Performance Evaluations of LLM Inference Frameworks

Eray Yüklü & Süleyman Tolga Acar
Advisor:Atay Özgövde

Introduction

Deploying advanced agentic workflows at scale using commercial LLM APIs is financially prohibitive, while budget-friendly self-hosted models often lack the reasoning capacity for complex tasks. To bridge this gap, this project first optimizes self-hosted LLM serving on a single NVIDIA L4 GPU through systematic vLLM parameter sweeps, evaluating latency, throughput, and memory metrics alongside standard quality benchmarks. We then implement a hybrid Planner-Executor agent architecture that combines the planning strengths of a commercial API with the low cost of our optimized self-hosted execution layer to achieve cost-efficient, high-accuracy task resolution.

Phase 1 Methodology: Serving Optimization

Parameter Optimization Sweeps: Swept combinations of model weight quantization (FP16, FP8, AWQ 4-bit), KV cache precision formats (FP16, FP8, and TurboQuant levels), memory manager limits, and scheduling features (chunked prefill, prefix caching).

L**ocust Load Testing & Serving Metrics:** Simulated concurrent user profiles ranging from 16 to 128 active requests using distributed Locust with the ShareGPT dataset, capturing diverse prompt types and multi-turn chat histories, to measure latency (TTFT, TPOT, ITL, and queue delays), token throughput (input/output tokens per second), and hardware telemetry (power usage, VRAM footprint).

Model Quality Benchmarking: Executed accuracy evaluations using standardized benchmarks (ARC, GSM8K, and MMLU) to quantify and isolate any degradation in reasoning and problem-solving performance caused by quantization.

Phase 2 Methodology: Asymmetric Agent

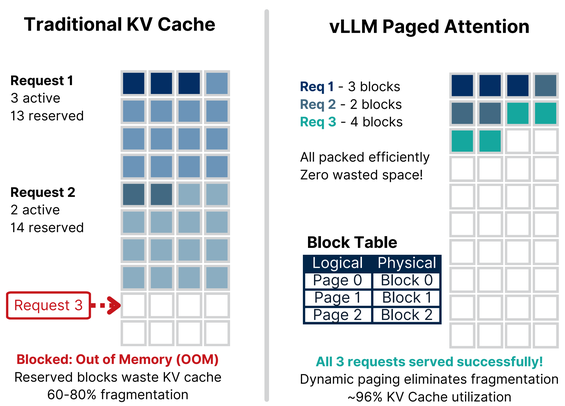
Asymmetric Agent Workflow: User queries are routed to a high-capability commercial model (the Planner) that generates a structured JSON execution plan containing discrete sub-tasks, verification checks, and tool calls. A cost-efficient self-hosted model (the Executor) then executes this plan sequentially.

Evaluated Configurations:

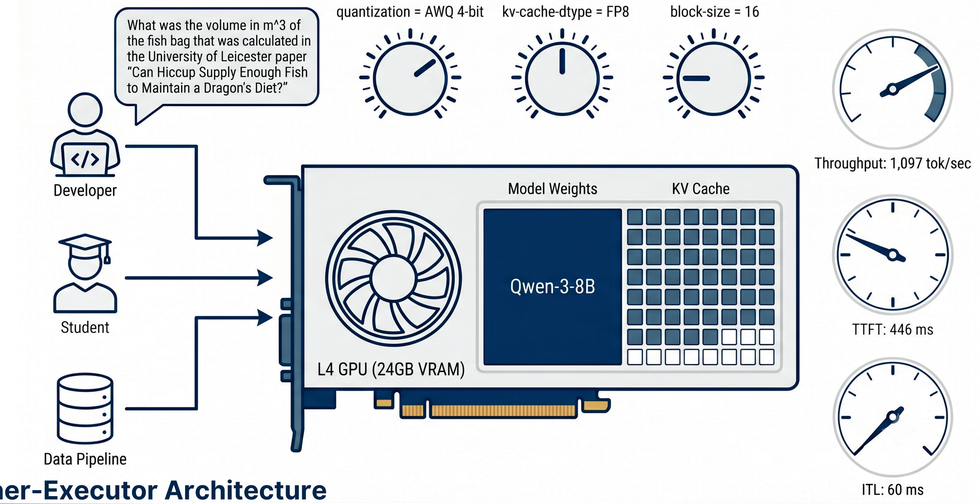
- Planner: Gemini 3 Flash / Executor: Gemini 3 Flash (API-only baseline)
- Planner: Gemini 3 Flash / Executor: Gemma 4 26B (Proposed hybrid setup)
- Planner: Gemma 4 26B / Executor: Gemma 4 26B (Fully self-hosted setup)
- Gemini 3 Flash (Single API model agent)
- Gemma 4 26B (Single self-hosted model agent)

Tool Suite & Evaluation: The Executor has access to tools for web searching, web scraping, multi-format document parsing, and sandboxed Python execution. All configurations were benchmarked on the GAIA Level 1 validation set (53 complex multi-modal questions) to measure accuracy, execution duration, and API cost.

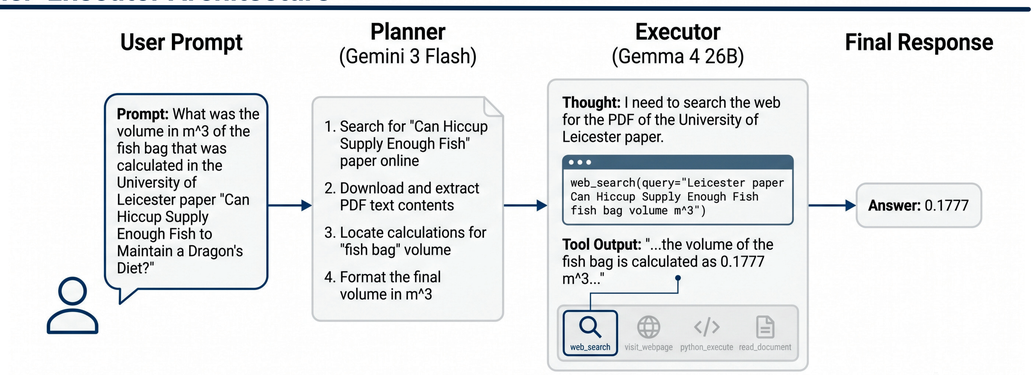
Paged Attention



vLLM Serving Optimization

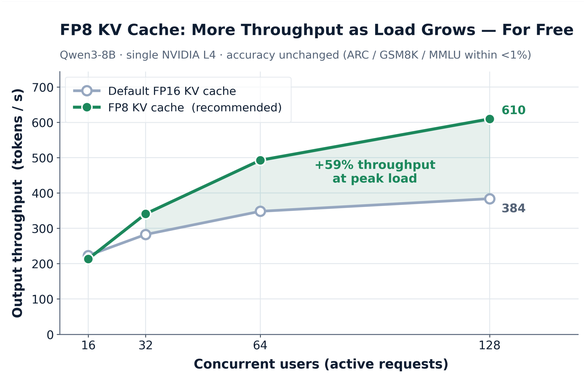


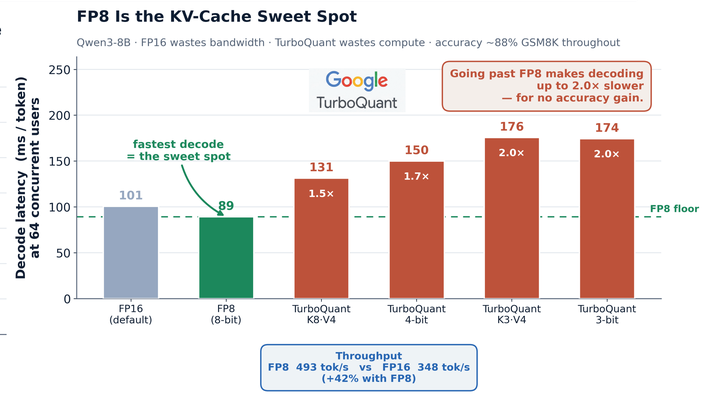
Planner-Executor Architecture

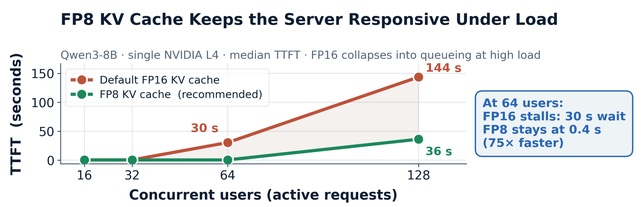


Accuracy on GAIA L1: 67.92% (Retains 80% of max capability)
Cost Savings: 90% ($0.011/Q vs. $0.121/Q)

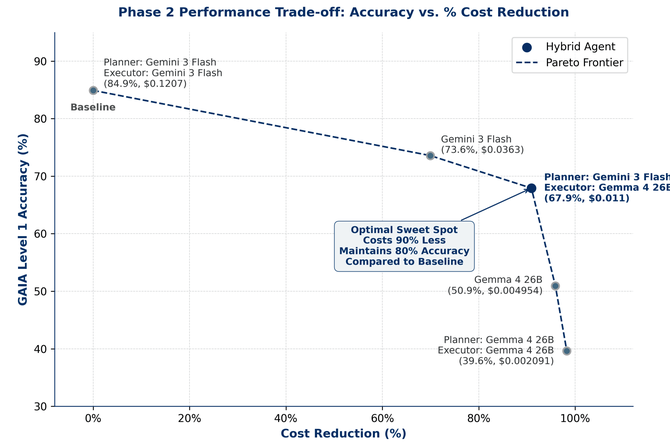
Phase 1 Results







Phase 2 Results



Conclusion

Optimal Inference Serving: Utilizing FP8 model weights paired with an FP8 KV cache represents the Pareto-optimal default configuration, maximizing serving throughput on single-GPU hardware with negligible accuracy loss. Additionally, chunked prefill and prefix caching act as cost-free performance wins that should always be enabled.

TurboQuant Hardware Limits: TurboQuant's optimization value is heavily hardware-bound; when serving an 8B model on the NVIDIA L4 GPU, its dequantization compute overhead completely overwhelms the VRAM and memory bandwidth savings of tighter KV cache compression.

Asymmetric Agent Architecture: While structured planning consistently improves task-solving accuracy across all agent modes, pairing a larger frontier model for planning with a smaller, highly optimized self-hosted model for execution forms the Pareto-optimal agent architecture, matching commercial API quality at a fraction of the cost.

Future Work

Dynamic Task Routing: Implement a lightweight router to send simple queries directly to the self-hosted Executor, bypassing the Planner API to minimize costs.

Executor Fine-Tuning: Distill reasoning patterns and tool-use traces from the Planner into the local Executor (Gemma 4) to bridge the 40% accuracy gap.

Hardware Scaling: Test aggressive KV-cache compressions (TurboQuant) on newer GPUs (H100/H200) to eliminate the dequantization compute bottleneck.

References

vLLM (PagedAttention): Kwon et al. (2023). "Efficient Memory Management for Large Language Model Serving with PagedAttention." SOSP '23.
FrugalGPT: Chen et al. (2023). "FrugalGPT: How to Use Large Language Models Faster and Cheaper." arXiv preprint arXiv:2305.05176.